EXPERIMENT: MISSIONARIES AND CANNIBAL PROBLEM

Aim

To write a Python program to solve the Missionaries and Cannibal problem using Artificial Intelligence techniques.

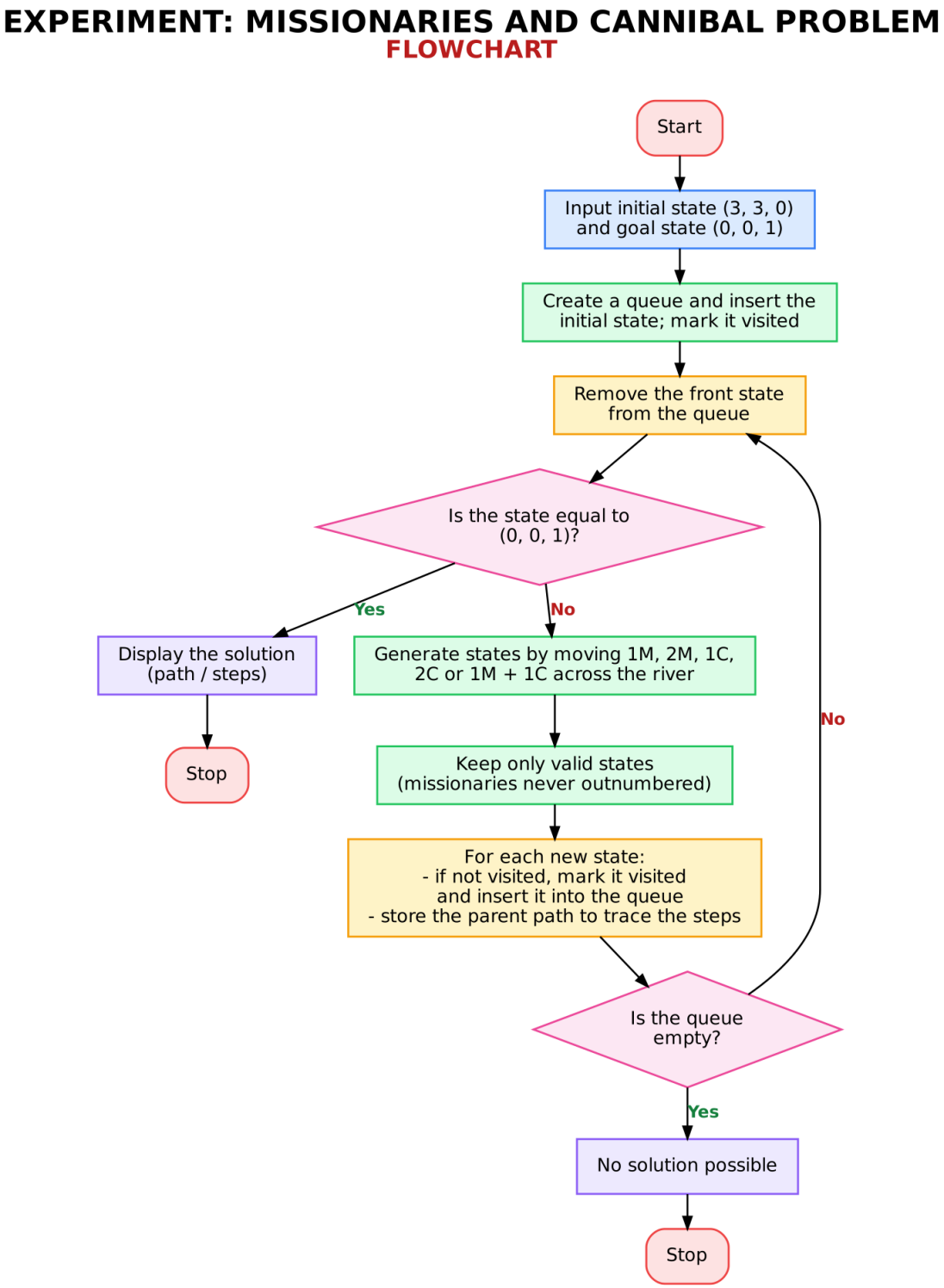
Objective

- To understand the state-space representation of the Missionaries and Cannibal problem.
- To implement Breadth First Search to find the sequence of river crossings.
- To learn how constraints (missionaries must never be outnumbered) are applied while generating states.

Algorithm

1. Start.
2. Represent a state as (missionaries on left bank, cannibals on left bank, boat side), where boat side 0 = left and 1 = right.
3. Define the initial state (3, 3, 0) and the goal state (0, 0, 1).
4. Insert the initial state into a queue.
5. Remove the front state from the queue.
6. If it is the goal state, display the path and stop.
7. Generate new states by moving 1 missionary, 2 missionaries, 1 cannibal, 2 cannibals or 1 missionary and 1 cannibal across the river.
8. Reject a state if cannibals outnumber missionaries on either bank (when missionaries are present) or the counts are invalid.
9. Add every valid and unvisited state to the queue and repeat from step 5.
10. Stop.

Flowchart:



Code (Python)

```
from collections import deque

# State = (missionaries on left, cannibals on left, boat side)  boat: 0 = left, 1 = right
TOTAL_M = 3
TOTAL_C = 3
def is_valid(m, c):
```

```
    if m < 0 or c < 0 or m > TOTAL_M or c > TOTAL_C:
        return False
    # missionaries must not be outnumbered on either bank (unless there are none)
    if m > 0 and m < c:
        return False
    if (TOTAL_M - m) > 0 and (TOTAL_M - m) < (TOTAL_C - c):
        return False
    return True

def get_next_states(state):
    m, c, boat = state
    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
    next_states = []
    for dm, dc in moves:
        if boat == 0:                       # boat moves left -> right
            new_state = (m - dm, c - dc, 1)
        else:                               # boat moves right -> left
            new_state = (m + dm, c + dc, 0)
        if is_valid(new_state[0], new_state[1]):
            next_states.append((new_state, (dm, dc)))
    return next_states

def solve():
    start = (TOTAL_M, TOTAL_C, 0)
    goal = (0, 0, 1)
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path
        for next_state, _ in get_next_states(state):
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [next_state]))
    return None

path = solve()

print("State format: (Missionaries left, Cannibals left, Boat side)")
print()
for step, state in enumerate(path):
    side = "Left" if state[2] == 0 else "Right"
    print("Step", step, ":", state, "- Boat on", side)
print()
print("Solved in", len(path) - 1, "crossings")
```

Output

Python Shell - Output

```
State format: (Missionaries left, Cannibals left, Boat side)

Step 0 : (3, 3, 0) - Boat on Left
Step 1 : (3, 1, 1) - Boat on Right
Step 2 : (3, 2, 0) - Boat on Left
Step 3 : (3, 0, 1) - Boat on Right
Step 4 : (3, 1, 0) - Boat on Left
Step 5 : (1, 1, 1) - Boat on Right
Step 6 : (2, 2, 0) - Boat on Left
Step 7 : (0, 2, 1) - Boat on Right
Step 8 : (0, 3, 0) - Boat on Left
Step 9 : (0, 1, 1) - Boat on Right
Step 10 : (1, 1, 0) - Boat on Left
Step 11 : (0, 0, 1) - Boat on Right

Solved in 11 crossings
```

Output Result

Thus, the Python program to solve the Missionaries and Cannibal problem was executed successfully, and all of them were moved to the right bank safely in 11 crossings.

Conclusion

The experiment helped in understanding state-space search with constraints. The Missionaries and Cannibal problem was solved successfully using Breadth First Search by generating only the safe states.